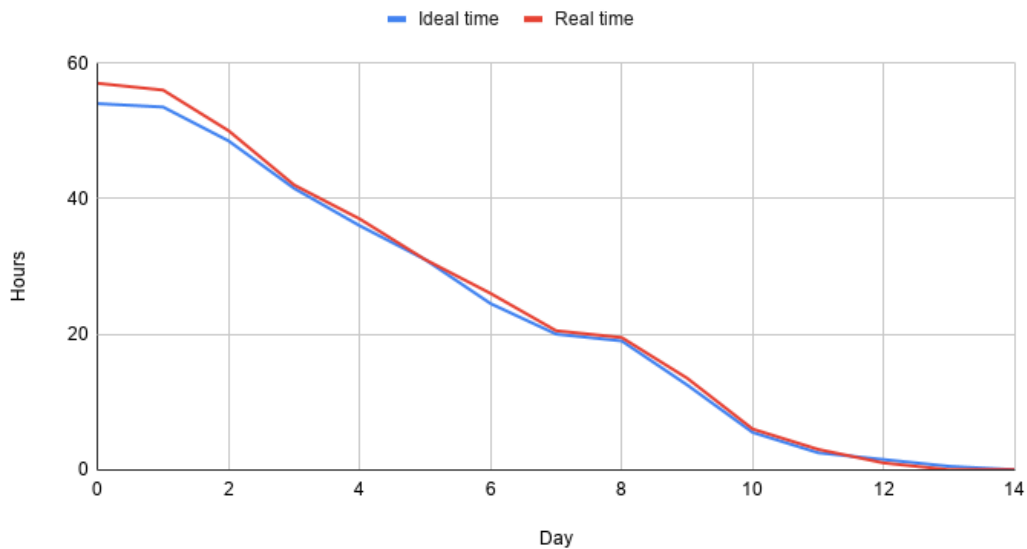


Sprint 2 - Review

Implemented features/completed work + design decisions (narrative)

- Database design (schema and E-R diagram)
- Web scraping for current and historical data
 - Error handling and inserting data into the new database
- Implementing FLASK for live/stored bike and weather data
- Preliminary website design
- Test live API data retrieval through EC2

Burndown chart



Burndown chart for Sprint 2: This chart shows that we started the sprint slightly behind the ideal pace, which reflects the amount of setup and infrastructure work that needed to be done early on and the fact that some initial estimates proved to be slightly optimistic in terms of time taken to finish the task. As the sprint progressed we maintained a consistent pace, tracking closely to the ideal pace throughout. By the end of the two weeks we had completed all of the tasks we set out to do.

Sprint review

Sprint 2 focused on transitioning from the planning phase into the implementation stage. The primary goals for this sprint focused on database design, the Flask application framework, developing a preliminary frontend and deployment.

In relation to data, web scraping was implemented for both bike availability and weather data, with bike station data collected at five-minute intervals and the weather data was collected across current, hourly and daily endpoints. A database schema was designed and implemented to store scraped bike and weather data, and an Entity-Relationship diagram was produced to

document the design. Error handling, logging and data retention cleanup processes were also put in place to ensure the data was both reliable and manageable. The required data fields were identified and JSON responses from both APIs were transformed into structured data before being stored in the database.

The Flask application skeleton was created during this sprint, with endpoints established for live and stored bike data, live and stored weather data and the initial display of them both on the preliminary web page. Bike stations were rendered on the map using live data, providing the first visual representation of the core functionality for this application. A Git workflow was also formalised during this sprint, including a pull request template and a feature branch process. This was done to ensure that collaboration remained clean for when the repository became more complex in future sprints. Additionally, a preliminary frontend was created using hardcoded scraped data to visualise the general layout before integration in a later sprint.

Sprint retrospective

Overall, this was a productive sprint where we successfully set up the core infrastructure of the project, including the database, Flask application and EC2 deployment which were the most important elements for this sprint. Having these foundations in place early on gave us confidence going into the next sprint. Formalising the Git workflow and pull request process also made collaboration cleaner and reduced the risk of overlapping work as the repository grew in size. Progress was slower than we initially thought, particularly in the first week of the sprint. Some of the estimates in the backlog proved to be optimistic, especially for the database implementation and deployment tasks which took longer than expected. As a result of this we were consistently behind the ideal burndown pace throughout the sprint, but only slightly and our work pace remained fairly consistent. For sprint 3 we aim to be more realistic with our time estimates for sprint tasks, particularly for the tasks that involve integration and deployment.